Foundations: quantum-information preliminaries, number model, exact admissibility and emission

Statements digest, version 0.12.0

Every definition, fact, assumption, result and requirement of the contract, with the contract's numbers; remarks, examples, technical steps and all proofs are left out. Each heading opens its statement in the contract.

Mathematics

1 Quantum-information preliminaries

Definition 1.1 (Operator conventions and pinching)**.** All Hilbert spaces are finite-dimensional. For a system label A, $\mathcal{L}(A)$ is the space of linear operators on $\mathcal{H}(A)$, $\mathrm{Pos}(A)$ its positive semidefinite operators, and $S_{=}(A)$ and $S_{\leq}(A)$ the normalized and the subnormalized states, of trace equal to one and at most one. A channel is a CPTP map; an instrument branch is a CP trace-non-increasing (CPTNI) map. For $X \in \mathrm{Pos}(A)$ and $p > 0$, the power X^p is defined by functional calculus on the support of X and by $0^p = 0$ on its kernel; in particular X^p is block-diagonal for any orthogonal decomposition for which X is.

For a family of orthogonal projectors $\{\Pi_j\}_j$ on $\mathcal{H}(Q)$ with $\sum_j \Pi_j = \mathbb{1}_Q$, the associated *pinching* is the channel $\tau \mapsto \sum_j \Pi_j \tau \Pi_j$. A pinching is CPTP, self-adjoint as a superoperator ($\mathrm{tr}[Y\,\mathcal{E}(W)] = \mathrm{tr}[\mathcal{E}(Y)\,W]$), idempotent, and unital. An operator X is a fixed point iff it is block-diagonal, $X = \sum_j \Pi_j X \Pi_j$; functional calculus of a block-diagonal PSD operator acts blockwise, so a pinching commutes with taking powers of its fixed points. Each of these properties is a one-line computation from the form $\sum_j \Pi_j(\cdot)\Pi_j$ and orthogonality of the Π_j; self-adjointness is $\mathrm{tr}\Big[Y \sum_j \Pi_j W \Pi_j\Big] = \mathrm{tr}\Big[\sum_j \Pi_j Y \Pi_j\, W\Big]$ by cyclicity.

A pinching is a uniform mixture of unitary conjugations. Enumerate the projectors as $\Pi_1, \ldots, \Pi_m$, put $\zeta := e^{2\pi i/m}$ and $U_k := \sum_{j=1}^{m} \zeta^{jk} \Pi_j$ for $k \in \{0, \ldots, m-1\}$; each U_k is unitary, $U_k U_k^\dagger = \sum_j |\zeta^{jk}|^2 \Pi_j = \mathbb{1}_Q$ by orthogonality, and

$$\sum_j \Pi_j X \Pi_j = \frac{1}{m} \sum_{k=0}^{m-1} U_k X U_k^\dagger \qquad \text{for every } X \in \mathcal{L}(Q), \tag{1.1}$$

since the right-hand side is $\sum_{j,j'} \big(\frac{1}{m}\sum_k \zeta^{(j-j')k}\big) \Pi_j X \Pi_{j'}$ and the inner average is 1 for $j = j'$ and 0 otherwise. In particular a pinching does not

increase the operator norm, a mixture of unitary conjugates of X having norm at most $\|X\|$.

Definition 1.2 (Sandwiched Rényi divergence and conditional entropies)**.** For $\alpha \in (1,\infty)$, $\rho \in S_{\leq}(AB)$ and $\sigma \in \mathrm{Pos}(AB)$, on the same system,

$$\widetilde{D}_\alpha(\rho\,\|\,\sigma) \coloneqq \frac{1}{\alpha-1}\log_2 \mathrm{tr}\Big[\Big(\sigma^{\frac{1-\alpha}{2\alpha}}\,\rho\,\sigma^{\frac{1-\alpha}{2\alpha}}\Big)^\alpha\Big] \tag{1.2}$$

when $\mathrm{supp}\,\rho \subseteq \mathrm{supp}\,\sigma$, and $\widetilde{D}_\alpha(\rho\|\sigma) \coloneqq +\infty$ otherwise. The exponent $\frac{1-\alpha}{2\alpha}$ is negative, and the power of σ is taken on its support and is zero on its kernel, as for the generalised inverse; where $\mathrm{supp}\,\rho \subseteq \mathrm{supp}\,\sigma$, $\rho = P\rho P$ for the projector P onto that support, so the value reads the power on the support alone. For $\rho_{AB} \in S_=(AB)$,

$$\widetilde{H}^{\downarrow}_\alpha(A|B)_\rho \coloneqq -\widetilde{D}_\alpha(\rho_{AB}\,\|\,\mathbb{1}_A\otimes\rho_B), \qquad \widetilde{H}^{\uparrow}_\alpha(A|B)_\rho \coloneqq -\inf_{\sigma_B\in S_=(B)} \widetilde{D}_\alpha(\rho_{AB}\,\|\,\mathbb{1}_A\otimes\sigma_B). \tag{1.3}$$

Both are in bits. These are the definitions of [1, Def. 5.2, Eqs. (5.18)–(5.19)]. Since the infimum in (1.3) includes $\sigma_B = \rho_B$,

$$\widetilde{H}^{\downarrow}_\alpha(A|B)_\rho \;\leq\; \widetilde{H}^{\uparrow}_\alpha(A|B)_\rho. \tag{1.4}$$

The substitution $\widetilde{H}^{\uparrow}_\alpha \to \widetilde{H}^{\downarrow}_\alpha$ inside a lower bound is therefore conservative, and the reverse is not.

Definition 1.3 (Classical divergences and the quantum relative entropy)**.** For probability vectors $\mathbf{q}, \mathbf{p}$ on a finite alphabet, $D(\mathbf{q}\|\mathbf{p}) \coloneqq \sum_c q(c)\log_2\frac{q(c)}{p(c)}$ with the conventions $0\log_2\frac{0}{p} = 0$ and $q\log_2\frac{q}{0} = +\infty$ for $q > 0$. For $\alpha \in (1,\infty)$ the classical Rényi divergence is $D_\alpha(\mathbf{q}\|\mathbf{p}) \coloneqq \frac{1}{\alpha-1}\log_2\sum_c q(c)^\alpha p(c)^{1-\alpha}$ (with $+\infty$ if $\mathbf{q} \not\ll \mathbf{p}$), and $D_\infty(\mathbf{q}\|\mathbf{p}) \coloneqq \log_2 \max_c \frac{q(c)}{p(c)}$. For $X, Y \in \mathrm{Pos}()$ the quantum relative entropy is $D(X\|Y) = \mathrm{tr}[X(\log_2 X - \log_2 Y)]$ when $\mathrm{supp}\,X \subseteq \mathrm{supp}\,Y$, else $+\infty$.

The Kullback–Leibler divergence of probability vectors is non-negative, $D(\mathbf{q}\|\mathbf{p}) \geq 0$, with equality if and only if $\mathbf{q} = \mathbf{p}$ (Gibbs' inequality). If $q(c) > 0 = p(c)$ for some c, then $D(\mathbf{q}\|\mathbf{p}) = +\infty$. Otherwise, with $S \coloneqq \{c : q(c) > 0\}$, the terms outside S vanish by the first convention and $\log_2$ is strictly concave, so Jensen's inequality with the weights $q(c)$, $c \in S$, which sum to one, gives $-D(\mathbf{q}\|\mathbf{p}) = \sum_{c\in S} q(c)\log_2\frac{p(c)}{q(c)} \leq \log_2\sum_{c\in S} p(c) \leq \log_2 1 = 0$. Equality in the first step forces $p(c)/q(c)$ to be one constant γ on S, and in the second $\sum_{c\in S} p(c) = 1$; then $\gamma = \gamma\sum_{c\in S} q(c) = \sum_{c\in S} p(c) = 1$, so $\mathbf{p} = \mathbf{q}$ on S, and $\mathbf{p}$ vanishes off S, its mass on S being one. Conversely $D(\mathbf{q}\|\mathbf{q}) = 0$ term by term.

Lemma 1.4 (Extended divergence kernel). Define $d : [0,\infty)\times[0,\infty) \to (-\infty,+\infty]$ by $d(u,v) := u\log_2 \frac{u}{v}$ for $u,v>0$, $d(0,v) := 0$ for $v \geq 0$, and $d(u,0) := +\infty$ for $u>0$. Then d is jointly convex and lower semicontinuous, and non-increasing in v.

Lemma 1.5 (Pinching lowers the trace power). Let $\tau \in \mathrm{Pos}(Q)$, let $\mathcal{Z}$ be a pinching with projectors $\{Z_j\}_{j=1}^m$, and let $\alpha \in (1,\infty)$. Then

$$\mathrm{tr}\Big[\Big(\mathcal{Z}\big(\tau^{1/\alpha}\big)\Big)^{\alpha}\Big] \;\leq\; \mathrm{tr}[\tau], \tag{1.5}$$

with equality if and only if $\mathcal{Z}(\tau^{1/\alpha}) = \tau^{1/\alpha}$, equivalently iff τ is block-diagonal for $\{Z_j\}$, equivalently iff $\mathcal{Z}(\tau) = \tau$.

Fact 1.6 (Pinched trace-power form of the conditional Rényi entropy)**.** Let $\rho_{QE} \in S_=(QE)$ be pure, let $\{Z_j\}_{j=1}^d$ be orthogonal projectors on Q with $\sum_j Z_j = \mathbb{1}_Q$, let $V_Z = \sum_j |j\rangle_S \otimes Z_j$ and $\sigma_{SQE} := V_Z\,\rho_{QE}\,V_Z^\dagger$, and let $\mathcal{Z}(\tau) := \sum_j Z_j \tau Z_j$. Then, for $\alpha \in (1,\infty)$,

$$\widetilde{H}^{\downarrow}_{\alpha}(S|E)_{\sigma_{SE}} = \frac{1}{1-\alpha}\log_2 \mathrm{tr}\Big[\Big(\mathcal{Z}\big(\rho_Q^{1/\alpha}\big)\Big)^{\alpha}\Big]. \tag{1.6}$$

Locator note. The source states no α restriction in the theorem; the domain $(1,\infty)$ used here lies inside the domain $(0,1)\cup(1,\infty)$ of its entropy definitions and is the only range any claim that uses this fact invokes.

Fact 1.7 (Concavity of the pinched trace-power functional)**.** Let $\mathcal{Z}$ be a pinching built from a complete family of mutually orthogonal projectors (Def. 22 of the source) and for $\alpha' \in [0,2]$ let $Q_{\alpha'}(\rho) := \mathrm{tr}\Big[(\mathcal{Z}(\rho^{\alpha'}))^{1/\alpha'}\Big]$ (Eq. (F6) of the source). Then $\rho \mapsto Q_{\alpha'}(\rho)$ is concave on $S_{\leq}(Q)$ for $\alpha' \in [0,1]$.

2 The round model

Definition 2.1 (Round model, attack sequences and attack sets)**.** Fix a number $n \geq 1$ of rounds, a per-round system S, Alice's system A, and the instantiating model's source mode: in trusted-source mode Alice supplies A in every round, and in untrusted-source mode the adversary does. Systems, operators and channels follow the conventions of Definition 1.1, except that S may be separable of infinite dimension where the instantiating model declares it so; a channel whose input or output includes such a system is a normal CPTP map on trace-class operators. The dimensions are the instantiating model's to declare. We write S_j and A_j for the copies of S and A in round j, and drop the round index where one round is in view. This is the *round model* with per-round system S.

(i) An *attack sequence* is a family $(\mathcal{A}_j)_{j=1}^{n}$ of channels whose output carries the systems the adversary supplies in the source mode:

$$\mathcal{A}_j \in \begin{cases} \mathrm{CPTP}\big(E_{j-1},\ S_j\,E'_j\big), & \text{trusted-source mode,} \\ \mathrm{CPTP}\big(E_{j-1},\ A_j\,S_j\,E'_j\big), & \text{untrusted-source mode,} \end{cases} \tag{2.1}$$

where E_0 is Eve's initial register, E'_j her outgoing memory, and E_j her register after round j, which holds E'_j together with what the instantiating model hands her in round j. In trusted-source mode A_j is *not* an output of $\mathcal{A}_j$: it is supplied by Alice and carried into the round as a separate input.

(ii) Let S be finite-dimensional, and let $\mathfrak{L} = \{(\hat{O}_k, \bowtie_k, c_k)\}_{k=1}^{m}$, $m \geq 0$, be a finite list of Hermitian operators $\hat{O}_k$ on S, relations $\bowtie_k \in \{\preceq, =, \succeq\}$ between operators, and real numbers c_k. For registers E', E'', the *attack set* of $\mathfrak{L}$ is

$$\mathfrak{A}_{\mathfrak{L}}(E',\, SE'') \coloneqq \Big\{\mathcal{A} \in \mathrm{CPTP}(E', SE'') :\ \mathcal{A}^{\dagger}\big(\hat{O}_k \otimes \mathbb{1}_{E''}\big) \bowtie_k c_k\, \mathbb{1}_{E'}\ \ \forall k\Big\} \qquad \text{(trusted-source mode} \tag{2.2}$$

and, with the constraint operator lifted to the larger supplied system,

$$\mathfrak{A}_{\mathfrak{L}}(E',\, ASE'') \coloneqq \Big\{\mathcal{A} \in \mathrm{CPTP}(E', ASE'') :\ \mathcal{A}^{\dagger}\big(\mathbb{1}_A \otimes \hat{O}_k \otimes \mathbb{1}_{E''}\big) \bowtie_k c_k\, \mathbb{1}_{E'}\ \ \forall k\Big\} \tag{2.3}$$

in untrusted-source mode; the two are distinguished by the output system named in the argument. For $\mathfrak{L} = \varnothing$ the condition is empty and $\mathfrak{A}_{\varnothing} = \mathrm{CPTP}$.

(iii) Where S is finite-dimensional and $S = \bigotimes_{f \in \mathbb{F}} S_f$ is a product of factors indexed by a finite ordered set $\mathbb{F}$ (with a round index as well, a factor is written $S_{f,j}$), an entry of $\mathfrak{L}$ may be *tagged* with a factor f; its operator $\hat{O}_k$ is then Hermitian on S_f, and the entry $(\hat{O}_k, \bowtie_k, c_k; f)$ contributes to the attack set, in place of the condition of (2.2) or (2.3), the condition

$$\big(\mathrm{id}_{R'} \otimes \mathcal{A}\big)^{\dagger}\Big(X \otimes \big(\hat{O}_k - c_k\, \mathbb{1}_{S_f}\big) \otimes \mathbb{1}_{E''}\Big)\ \bowtie_k\ 0 \qquad \text{for every finite-dimensional } R' \text{ and every } X \in \mathrm{P} \tag{2.4}$$

as an operator on $R'E'$, where S_{f^c} is the product of the factors other than f, the tensor factors of the argument are written in the order $R'R\,S_{f^c},\ S_f,\ E''$, and R is A in untrusted-source mode and trivial in trusted-source mode. Untagged entries keep their conditions. The reference system R' is what carries Alice's retained system in trusted-source mode, where A is no output of $\mathcal{A}$. With R' trivial and $X = \mathbb{1}$, (2.4) is the condition of (2.2) or (2.3) for the lifted operator $\mathbb{1}_{S_{f^c}} \otimes \hat{O}_k$, because $\mathcal{A}^{\dagger}$ is unital; so the attack set with tagged entries is contained in the attack set with the same entries untagged and their operators lifted.

(iv) An attack sequence is an *admissible attack sequence* for $\mathfrak{L}$, or $\mathfrak{L}$-*admissible*, if $\mathcal{A}_j$ lies in the attack set of $\mathfrak{L}$ of the source mode, (2.2) or (2.3) with the conditions of (2.4) for tagged entries, with $E' = E_{j-1}$ and $E'' = E'_j$, for every j.

Definition 2.2 (Round homogeneity of a per-round family)**.** Consider the n rounds of a round model (Definition 2.1), and a *per-round family*: a finite list of objects that a model of the run declares for a round, such as operators, channels or a declaration. The family is *round-homogeneous* if both of the following hold.

(i) *Sameness.* One list of objects, fixed before the run, describes every round: for every $j \in \{1, \ldots, n\}$ round j is described by the same objects. No object of the family depends on the round index or on the run.

(ii) *No memory.* Every round behaves physically as the objects describe it, whatever the inputs and the outputs of the other rounds are. This excludes memory across rounds, such as detector memory, afterpulsing or correlations between a source's emissions in different rounds. It is a condition on the rounds, not on the objects, and does not follow from (i).

An assumption may assert (i) alone of its family, and must then say so; an assumption that calls its family round-homogeneous without saying so asserts (i) and (ii). Rounds are not averaged: a family that describes the rounds only on average over the rounds, or up to a per-round variation, does not satisfy (i), and per-round variation is not modelled. Round homogeneity constrains the family and the rounds it describes, not the adversary: an attack sequence of the round model remains arbitrary and may correlate the rounds.

Lemma 2.3 (Compressing a register onto a finite-dimensional subspace)**.** *Setting.* In this lemma a Hilbert space may have any dimension, finite or infinite, separable or not, in departure from the convention of Definition 1.1, and every system named is of this kind. $\mathcal{T}(Z)$ is the space of trace-class operators on $\mathcal{H}(Z)$ with the trace norm $\|\cdot\|_1$, and RM is the Hilbert-space tensor product of R and M. A *channel* $\mathcal{T}(Z) \to \mathcal{T}(Z')$ is a trace-preserving linear map Φ that is completely positive: for every $k \geq 1$ the map that applies Φ to every entry of a $k \times k$ matrix of trace-class operators, which is $\mathcal{T}(\mathbb{C}^k Z)$, is positive. $\mathrm{tr}_R \colon \mathcal{T}(RM) \to \mathcal{T}(M)$ is the partial trace, fixed by $\mathrm{tr}[X \,\mathrm{tr}_R(\sigma)] = \mathrm{tr}[(\mathbb{1}_R \otimes X)\sigma]$ for every bounded operator X on $\mathcal{H}(M)$, and tr_M likewise.

(i) *(compression and inclusion)* Let M° be a non-zero finite-dimensional subspace of $\mathcal{H}(M)$, V its inclusion isometry into $\mathcal{H}(M)$, $\Pi \coloneqq VV^\dagger$ the orthogonal projection onto it, $\Pi^\perp \coloneqq \mathbb{1}_M - \Pi$, and ψ a state on M°. For every system R put, for $\sigma \in \mathcal{T}(RM)$ and $\tau \in \mathcal{T}(RM^\circ)$,

$$\begin{gathered}\mathcal{P}_R(\sigma) \coloneqq (\mathbb{1}_R \otimes V)^\dagger \sigma (\mathbb{1}_R \otimes V) + \mathrm{tr}_M \left[(\mathbb{1}_R \otimes \Pi^\perp)\, \sigma\, (\mathbb{1}_R \otimes \Pi^\perp)\right] \otimes \psi, \\ \iota_R(\tau) \coloneqq (\mathbb{1}_R \otimes V)\, \tau\, (\mathbb{1}_R \otimes V)^\dagger,\end{gathered} \tag{2.5}$$

and write $\mathcal{P}$ and ι for the case of a one-dimensional R. Then $\mathcal{P}_R \colon \mathcal{T}(RM) \to \mathcal{T}(RM^\circ)$ and $\iota_R \colon \mathcal{T}(RM^\circ) \to \mathcal{T}(RM)$ are channels, $\mathcal{P}_R \circ \iota_R$ is the identity, $\|\iota_R(\tau)\|_1 = \|\tau\|_1$ for every τ, and $\mathcal{P}_R(x \otimes y) = x \otimes \mathcal{P}(y)$ and $\iota_R(x \otimes z) = x \otimes \iota(z)$ for $x \in \mathcal{T}(R)$, $y \in \mathcal{T}(M)$ and $z \in \mathcal{T}(M^\circ)$; more generally, for $R = R_1R_2$, $\mathcal{P}_{R_1R_2}(x \otimes y) = x \otimes \mathcal{P}_{R_2}(y)$ and $\iota_{R_1R_2}(x \otimes z) = x \otimes \iota_{R_2}(z)$ for $x \in \mathcal{T}(R_1)$, $y \in \mathcal{T}(R_2M)$ and $z \in \mathcal{T}(R_2M^\circ)$. The channel $\mathcal{C}_R \coloneqq \iota_R \circ \mathcal{P}_R$ on $\mathcal{T}(RM)$ is the *compression of M onto M°*.

(ii) *(one compression)* For every state σ on RM, with $\delta \coloneqq \mathrm{tr}\left[(\mathbb{1}_R \otimes \Pi^\perp)\sigma\right]$,

$$\left\|\mathcal{C}_R(\sigma) - \sigma\right\|_1 \;\leq\; 2\sqrt{\delta} + \delta. \tag{2.6}$$

For every state σ on RM and every $\eta > 0$ there is a non-zero finite-dimensional subspace M°, spanned by finitely many eigenvectors of $\mathrm{tr}_R(\sigma)$, at which $\delta \leq \eta$.

(iii) *(a sequence of channels)* Let $Z_0, \ldots, Z_m$ be systems, ω a state on Z_0, $\Phi_k \colon \mathcal{T}(Z_{k-1}) \to \mathcal{T}(Z_k)$ channels for $k = 1, \ldots, m$, and $\mathcal{C}_k$ a channel on $\mathcal{T}(Z_k)$ for $k = 0, \ldots, m$. Put $\sigma_0 \coloneqq \omega$ and $\sigma_k \coloneqq \Phi_k(\sigma_{k-1})$. Then

$$\left\|\mathcal{C}_m \Phi_m \mathcal{C}_{m-1} \cdots \Phi_1 \mathcal{C}_0(\omega) - \sigma_m\right\|_1 \;\leq\; \sum_{k=0}^{m} \left\|\mathcal{C}_k(\sigma_k) - \sigma_k\right\|_1. \tag{2.7}$$

(iv) *(maps on different factors)* Let $G \colon \mathcal{T}(R) \to \mathcal{T}(R')$ and $H \colon \mathcal{T}(M) \to \mathcal{T}(M')$ be linear maps, and let $G_M \colon \mathcal{T}(RM) \to \mathcal{T}(R'M)$, $G_{M'} \colon \mathcal{T}(RM') \to \mathcal{T}(R'M')$, $H_R \colon \mathcal{T}(RM) \to \mathcal{T}(RM')$ and $H_{R'} \colon \mathcal{T}(R'M) \to \mathcal{T}(R'M')$ be linear maps, bounded in trace norm, with $G_M(x \otimes y) = G(x) \otimes y$, $G_{M'}(x \otimes y') = G(x) \otimes y'$, $H_R(x \otimes y) = x \otimes H(y)$ and $H_{R'}(x' \otimes y) = x' \otimes H(y)$ for all $x \in \mathcal{T}(R)$, $x' \in \mathcal{T}(R')$, $y \in \mathcal{T}(M)$, $y' \in \mathcal{T}(M')$. Then $H_{R'} \circ G_M = G_{M'} \circ H_R$. In particular, with $H = \mathcal{P}$ or $H = \iota$, whose extensions are $\mathcal{P}_R$, $\mathcal{P}_{R'}$, ι_R and $\iota_{R'}$ by (i),

$$\mathcal{P}_{R'} \circ G_M = G_{M^\circ} \circ \mathcal{P}_R, \qquad G_M \circ \iota_R = \iota_{R'} \circ G_{M^\circ}. \tag{2.8}$$

(v) *(extension by an identity)* For every channel $\Phi \colon \mathcal{T}(Z) \to \mathcal{T}(Z')$ and every system K there is exactly one linear map $\Phi_K \colon \mathcal{T}(ZK) \to \mathcal{T}(Z'K)$, bounded in trace norm, with $\Phi_K(x \otimes y) = \Phi(x) \otimes y$ for all $x \in \mathcal{T}(Z)$ and $y \in \mathcal{T}(K)$; it is a channel, written $\Phi \otimes \mathrm{id}_K$.

3 Binary64 numbers and exact sums

Definition 3.1 (Binary64 numbers, declared values and dyadic lattices). (i) Let $\mathbb{B}$ be the set of real numbers $m\,2^q$ with $m, q \in \mathbb{Z}$, $|m| < 2^{53}$ and $-1074 \le q \le 971$, and let $\mathbb{B}_{\mathbb{C}} := \{x+iy : x, y \in \mathbb{B}\}$. The elements of $\mathbb{B}$ are the *binary64 numbers*, the finite values of the IEEE 754 binary64 format. Every binary64 number is a dyadic rational, an element of $\mathbb{Z}[\frac{1}{2}]$.

(ii) For a real number x with $|x| < 2^{1024} - 2^{970}$, $\mathrm{fl}(x)$ is x rounded to the nearest element of $\mathbb{B}$, a tie going to the element whose m is even in its representation of (i) with the least q. For $|x| \ge 2^{1024} - 2^{970}$, $\mathrm{fl}(x) := +\infty$ if $x > 0$ and $-\infty$ if $x < 0$; the threshold is the midpoint between the largest element $(2^{53}-1)\,2^{971}$ of $\mathbb{B}$ and 2^{1024}, which is where IEEE 754 rounding to nearest overflows. Further, $\mathrm{fl}(\pm\infty + y) := \pm\infty$ for $y \in \mathbb{B}$. The *binary64 sum* of a and b is $\mathrm{fl}(a+b)$. On $\mathbb{B}_{\mathbb{C}}$, and on matrices over it, fl acts on the real and on the imaginary part of every entry separately. Two properties of fl are all that is used of it: on real input its value lies in $\mathbb{B}\cup\{\pm\infty\}$, and $\mathrm{fl}(x) = x$ for $x \in \mathbb{B}$. The binary64 sum of a and b is *exact* when it equals $a+b$; by those two properties it is exact if and only if the real and the imaginary part of every entry of $a+b$ lie in $\mathbb{B}$.

(iii) A *declared value* is a binary64 number stated as a datum. It denotes that dyadic rational and nothing else: there is no exact object behind it of which it is an approximation, and every predicate stated of declared values is a predicate of those rationals. A declared complex value is an element of $\mathbb{B}_{\mathbb{C}}$, and a declared matrix is a matrix of declared complex values.

(iv) A *lattice exponent* is an integer k with $1 \le k \le 53$. A real number x is a *lattice value* at k when $2^k x \in \mathbb{Z}$ and $|x| \le 1$; a complex number, or a matrix, is one when the real and the imaginary part of every entry are.

Lemma 3.2 (Binary64 numbers on dyadic lattices). (i) For every lattice exponent k, every real lattice value at k is a binary64 number, and every complex lattice value at k lies in $\mathbb{B}_{\mathbb{C}}$.

(ii) For every integer e with $-1022 \le e \le 1023$,

$$\mathbb{B} \cap \left[2^e, 2^{e+1}\right) \;=\; 2^{e-52}\,\mathbb{Z} \cap \left[2^e, 2^{e+1}\right). \tag{3.1}$$

In particular the binary64 numbers in $[\frac{1}{2}, 1)$ are the multiples of 2^{-53} in that interval, and those in $[1, 2)$ are the multiples of 2^{-52} in it.

Definition 3.3 (Sums in a declared order and sums that round nowhere)**.** Let $(x_1, \ldots, x_n)$, $n \geq 1$, be a finite sequence of elements of $\mathbb{B}_{\mathbb{C}}$, or of matrices of one shape over $\mathbb{B}_{\mathbb{C}}$ (Definition 3.1 (i)): declared values and declared matrices (Definition 3.1 (iii)), or the computed inner sums of a sum of sums below. Its *sequential sum* is

$$S_0 := 0, \qquad S_i := S_{i-1} + x_i \quad (1 \leq i \leq n), \tag{3.2}$$

and its *computed sequential sum* is $\hat{S}_0 := 0$, $\hat{S}_i := \mathrm{fl}(\hat{S}_{i-1} + x_i)$ for $1 \leq i \leq n$ (Definition 3.1 (ii)), formed left to right, entrywise, with no re-ordering and no re-association. The accumulation *rounds nowhere* when every one of its n binary64 additions is exact: for every $1 \leq i \leq n$, every entry of $\hat{S}_{i-1}$ is finite and $\hat{S}_i = \hat{S}_{i-1} + x_i$.

A *declared order* on a finite alphabet J is a total order $j_1 \prec j_2 \prec \cdots \prec j_{|J|}$ on J, fixed together with J. For a family $\{a_j\}_{j \in J}$ of declared values, or of declared matrices of one shape, the *sum in the declared order* is the sequential sum of $(a_{j_1}, \ldots, a_{j_{|J|}})$, and the family's sum rounds nowhere when that accumulation does. The trace of a declared $d \times d$ matrix X is accumulated as the sequential sum of $(X_{11}, X_{22}, \ldots, X_{dd})$, in increasing row index. A sum of such sums over an alphabet with a declared order rounds nowhere when every inner accumulation rounds nowhere and the sequential sum, in that order, of the computed inner sums rounds nowhere; that sequential sum is then the computed sum of sums. Each computed inner sum is then over $\mathbb{B}_{\mathbb{C}}$: its last addition is exact, so it equals a sum of finite numbers, and it is a value of fl, which lies in $\mathbb{B} \cup \{\pm\infty\}$ entry by entry and part by part (Definition 3.1 (ii)). The outer sequential sum is therefore one of the sequences above.

Lemma 3.4 (Exactness of a sum that rounds nowhere). Let $(x_1, \ldots, x_n)$ be a finite sequence of elements of $\mathbb{B}_{\mathbb{C}}$, or of matrices of one shape over $\mathbb{B}_{\mathbb{C}}$, such as declared values or the computed inner sums of a sum of sums, with sequential sum $(S_i)_{i=0}^n$ and computed sequential sum $(\hat{S}_i)_{i=0}^n$ (Definition 3.3). The following are equivalent:

(a) the accumulation rounds nowhere;

(b) $\hat{S}_i = S_i$ for every $0 \leq i \leq n$;

(c) for every $1 \leq i \leq n$, the real and the imaginary part of every entry of S_i are binary64 numbers.

If they hold, the computed sum $\hat{S}_n$ is the exact sum $\sum_{i=1}^n x_i$.

4 Declared data and exact admissibility

Definition 4.1 (Declared data and the parties to a module boundary)**.** (i) An *instance* is the party that fixes one protocol's device data, protocol

data and run configuration. The data it states are *declared data*, and it is their *declarer*.

(ii) A module is a *supplier* when its own claims decide their conditions on the data it reads or emits in floating point, at slacks an instance declares to it, rather than exactly. A floating-point evaluation of a device model is an example. Its numerical output is not declared data until an instance states it as its own declared data, and the instance is then its declarer. Numbers of a supplier's output that reach another module in a run are declared by the instance that fixed the run: those it rounds and states, and those it passes on unchanged, whether in a declaration of its own or by handing the supplier's output on untouched.

(iii) A module other than a supplier that emits numerical data to another module as its own declaration, constructed from declared data it read, is a *producer* of those data; they are declared data, and the producer is their declarer. Declared data that a module passes on unchanged keep their declarer, and the module is not their producer. A scalar bound that a producer emits and that a consumer has replaced by its rounding to a binary64 number, in a direction that the producer's own claims license as weakening it, is read, at its replaced value, as its producer's declaration of that bound, and keeps its producer as declarer; a set defined by such bounds is then declared data by (v).

(iv) A module other than a supplier that reads declared data is a *consumer* of them. The passage of declared data from their declarer to a consumer is a *module boundary*.

(v) A set defined by declared data, such as a feasible set or an acceptance set, is declared data. An object that a module computes itself in the course of a computation, such as an iterate of an optimiser or the return of an oracle, is a *computed object* and is not declared data.

(vi) Every entry of declared operator data is a declared value (Definition 3.1 (iii)).

Definition 4.2 (Sector maps and block-list operators)**.** A *sector map* $\mathfrak{m}$ consists of two components: a block kind, which is a label naming what its blocks are, and a finite non-empty set S of block identifiers listed in a fixed order, each identifier $s \in \mathsf{S}$ with a dimension $d_s \geq 1$. Two sector maps *agree* when they have the same block kind and the same identifiers, in the same order, with the same dimensions. A system of dimension d without a declared sector decomposition carries the *one-block map* of dimension d: one identifier, with dimension d, and a block kind and an identifier that are the same for every such system, so that two one-block maps agree if and only if their dimensions are equal.

A *block list* against $\mathfrak{m}$ is a list $X = (X^{(s)})_{s\in\mathsf{S}}$ of complex $d_s \times d_s$ matrices. It denotes the operator $\bigoplus_{s\in\mathsf{S}} X^{(s)}$ on $\bigoplus_{s\in\mathsf{S}} \mathbb{C}^{d_s}$, and we identify the two: the list is the operator, and no entry outside the blocks exists to be written, so a block list is block diagonal by construction. A family of block lists against $\mathfrak{m}$ is a family $\{X_j\}_{j\in J}$ of such lists, $X_j^{(s)}$ being the block s of X_j.

Because the direct sum acts block by block, $X = X^\dagger$ if and only if $X^{(s)} = X^{(s)\dagger}$ for every s; $X \succeq 0$ if and only if $X^{(s)} \succeq 0$ for every s, since $\langle v, Xv\rangle = \sum_s \langle v_s, X^{(s)} v_s\rangle$ for $v = (v_s)_s$; and $\sum_{j\in J} X_j = c\,\mathbb{1}$ for a scalar c if and only if $\sum_{j\in J} X_j^{(s)} = c\,\mathbb{1}_{d_s}$ for every s.

Definition 4.3 (Exact admissibility of a declared operator family). Let $\mathfrak{m}$ be a sector map with blocks $s \in \mathsf{S}$ of dimensions d_s (Definition 4.2), let J be a finite non-empty alphabet with a declared order, let c be a declared value, and let $\{a_j\}_{j\in J}$ be a family of block lists against a sector map $\mathfrak{m}'$ whose entries are declared values (Definition 3.1 (iii)). The family is *exactly admissible* for $\mathfrak{m}$ with constant c when

(T) *(sector map)* $\mathfrak{m}'$ agrees with $\mathfrak{m}$;

(H) *(Hermiticity)* $a_j^{(s)} = a_j^{(s)\dagger}$ for every $j \in J$ and $s \in \mathsf{S}$;

(P) *(positivity)* $a_j^{(s)} \succeq 0$ for every $j \in J$ and $s \in \mathsf{S}$;

(C) *(completeness)* $\sum_{j\in J} a_j^{(s)} = c\,\mathbb{1}_{d_s}$ for every $s \in \mathsf{S}$, as an identity of the declared numbers;

(S) *(the sum rounds nowhere)* for every $s \in \mathsf{S}$, the sum of $\{a_j^{(s)}\}_{j\in J}$ in the declared order of J rounds nowhere (Definition 3.3).

The family is exactly admissible for $\mathfrak{m}$ when it is so with constant $c = 1$; it is then a positive operator-valued measure on $\bigoplus_s \mathbb{C}^{d_s}$, by the block-by-block reading of Definition 4.2. Since the block s of a block list against $\mathfrak{m}'$ is a square matrix of the dimension $\mathfrak{m}'$ gives s, (T) holds if and only if $\mathfrak{m}'$ has the block kind and the identifiers of $\mathfrak{m}$, in the same order, and every block $a_j^{(s)}$ is a $d_s \times d_s$ matrix, J being non-empty. A family on a system without a declared sector decomposition is read against the one-block map of that system's dimension, and (T) then reduces to the agreement of that dimension with the size of the family's matrices.

Each condition is a property of the declared values themselves and involves no threshold and no norm: (T) compares declared labels and dimensions, (H) and (C) are identities between exactly represented numbers, (P) is a property of matrices with dyadic entries, and (S) is a property of the binary64 additions that form the declared-order sum. Under (S) the computed sum in the declared order is the exact sum (Lemma 3.4), so (C) holds if and only if that computed sum equals $c\,\mathbb{1}_{d_s}$ on every block.

5 The declared tolerance and the snap

Definition 5.1 (The declared tolerance and the defects it bounds)**.** The *declared tolerance* is a binary64 number τ_{num} with $0 \le \tau_{\mathrm{num}} < 1$, stated by an instance in its run configuration (Definition 4.1). For a complex matrix Y we write $\|Y\|_{\max} := \max_{i,l} |Y_{il}|$ for its *entrywise maximum modulus*. The *defects* of data are the entrywise maximum moduli of the differences their predicates name:

(i) the *Hermiticity defect* $\left\|X - X^{\dagger}\right\|_{\max}$ of a square matrix X;

(ii) the *completeness residue* $\max_{s \in \mathsf{S}} \left\|\sum_{j \in J} a_j^{(s)} - c\,\mathbb{1}_{d_s}\right\|_{\max}$ of a family of block lists $\{a_j\}_{j \in J}$ against a sector map with blocks $s \in \mathsf{S}$ (Definition 4.2), read with a declared constant c;

(iii) the *column residue* $\max_l \left|\sum_i P_{il} - 1\right|$ of a real matrix P read as column-stochastic;

(iv) the *trace residue* $|\operatorname{tr} X - 1|$ of a square matrix X read as having unit trace, and the *sum residue* $\left|\sum_x p_x - 1\right|$ of a vector p read as a probability vector.

The sums here are exact sums. A square matrix X passes the *positivity floor* at τ_{num} when $\lambda_{\min}\big(\frac{1}{2}(X + X^{\dagger})\big) \ge -\tau_{\mathrm{num}}$. Data lie *within the declared tolerance* when each of their defects is at most τ_{num} and each of their blocks read as positive semidefinite passes the positivity floor at τ_{num}.

Lemma 5.2 (The positivity floor against entrywise closeness to the cone)**.** Let $d \ge 1$ and $\delta \ge 0$, and write $\|\cdot\|$ for the operator norm and $\|\cdot\|_{\max}$ for the entrywise maximum modulus of complex $d \times d$ matrices.

(i) $\|Y\|_{\max} \le \|Y\| \le d\,\|Y\|_{\max}$ for every complex $d \times d$ matrix Y.

(ii) If X is Hermitian and $\lambda_{\min}(X) \ge -\delta$, then $X + \delta\,\mathbb{1}_d \succeq 0$ and $\|(X + \delta\,\mathbb{1}_d) - X\|_{\max} = \delta$.

(iii) If X is Hermitian and $\|X - P\|_{\max} \le \delta$ for some $P \succeq 0$, then $\lambda_{\min}(X) \ge -d\delta$. The bound is attained: for the $d \times d$ matrix E with every entry equal to 1, the matrices $X := -\delta E$ and $P := 0$ satisfy $\|X - P\|_{\max} = \delta$ and $\lambda_{\min}(X) = -d\delta$.

Definition 5.3 (Reference of a measurement family, exact or enclosed)**.** Let $\mathfrak{m}$ be a sector map with blocks $s \in \mathsf{S}$ of dimensions d_s (Definition 4.2), let J be a finite non-empty alphabet with a declared order, and let $c > 0$ be a declared value (Definition 3.1 (iii)). A *entrywise enclosure* of a complex $d \times d$ matrix X is a pair (X_c, r) of a Hermitian $d \times d$ matrix X_c with entries

in $\mathbb{Q}(i) = \{x + iy : x, y \in \mathbb{Q}\}$, its *centre*, and rationals $r_{il} = r_{li} \geq 0$, its *radii*, with

$$\left|X_{il} - (X_c)_{il}\right| \;\leq\; r_{il} \qquad \text{for all } i, l. \tag{5.1}$$

A *reference* against $\mathfrak{m}$ over J with constant c is a family $G = \{G_j\}_{j \in J}$ of block lists against $\mathfrak{m}$ whose blocks are Hermitian and positive semidefinite, with

$$\sum_{j \in J} G_j^{(s)} \;=\; c\,\mathbb{1}_{d_s} \qquad \text{for every } s \in \mathsf{S}, \tag{5.2}$$

read together with an entrywise enclosure $\left(C_j^{(s)}, r_j^{(s)}\right)$ of every block $G_j^{(s)}$. It is an *exact reference* when every radius is 0; then $G_j^{(s)} = C_j^{(s)}$ has entries in $\mathbb{Q}(i)$, and its Hermiticity, positive semidefiniteness and completeness are properties of finitely many Gaussian rationals.

For a Hermitian $d_s \times d_s$ matrix Y with entries in $\mathbb{Q}(i)$, the pair $\left(C_j^{(s)} - Y, r_j^{(s)}\right)$ is an entrywise enclosure of $G_j^{(s)} - Y$, since the two differences in (5.1) are equal. A reference is an object of its own beside a family declared from it: the entries of the declared family are declared values, and each denotes exactly the dyadic rational it is (Definition 3.1 (iii)).

Definition 5.4 (Factor enclosure of a block)**.** Let $d \geq 1$ and $k \geq 0$ be integers. A *matrix box* of complex $d \times k$ matrices is a pair (L_c, t) of a $d \times k$ matrix L_c with entries in $\mathbb{Q}(i)$, its *centre*, and rationals $t_{im} \geq 0$, its *radii*; it *contains* a complex $d \times k$ matrix L when $|L_{im} - (L_c)_{im}| \leq t_{im}$ for all i, m. For $k = 0$ it contains the one $d \times 0$ matrix.

A *factor enclosure* of a complex $d \times d$ matrix X is a triple (L_c, t, W) of a matrix box (L_c, t) of complex $d \times k$ matrices and a diagonal $k \times k$ matrix W with positive rational diagonal entries, its *core*, such that

$$X \;=\; LWL^\dagger \qquad \text{for some } L \text{ that } (L_c, t) \text{ contains.} \tag{5.3}$$

The number k is its *number of columns*, a matrix L that the box contains is a *factor* of the triple, and the matrices $LWL^\dagger$ with L a factor are its *members*, X among them; for $k = 0$ the one member is the zero $d \times d$ matrix. Every member is Hermitian and positive semidefinite, since $x^\dagger LWL^\dagger x = \|W^{1/2}L^\dagger x\|^2$ for every $x \in \mathbb{C}^d$.

A reference G (Definition 5.3) *carries* a factor enclosure on a block $G_j^{(s)}$ when it is read, besides the entrywise enclosure $\left(C_j^{(s)}, r_j^{(s)}\right)$ of that block, with a factor enclosure of $G_j^{(s)}$; the block then satisfies both (5.1) and (5.3).

Definition 5.5 (Snapping declared data onto a lattice)**.** Let D be data a declarer is about to declare, a producer's exact construction of what it emits (Definition 4.1), or a reference of an operator family (Definition 5.3) — a

family of block lists against a sector map (Definition 4.2) over an alphabet with a declared order, a matrix or a vector — read with the conditions its declaration states of it, and let k be a lattice exponent (Definition 3.1 (iv)). For a family read as a measurement, those conditions are exact admissibility with its declared constant (Definition 4.3); every sum they read is read in its declared order (Definition 3.3). A *snap* of D at k is data D' of the same shape, over the same alphabet in the same order, every entry of which is a lattice value at k, and which satisfy those conditions exactly. Its *displacement* is the largest entrywise modulus of $D' - D$, taken over every block of every element (Definition 5.1), with D replaced by its centres where D is a reference.

An entry of D is *structurally zero* when it is known to be exactly zero, which is the following and nothing else: where D is given as numbers, an entry that is the number 0; where D is a reference, an entry whose centre and radius are both 0. An entry of a reference whose centre is 0 and whose radius is not is not structurally zero. The *structural support* of a block $D_j^{(s)}$ is the set of indices i such that row i or column i of $D_j^{(s)}$ holds an entry that is not structurally zero. A row, a column or an element all of whose entries are structurally zero is structurally zero, and an index lies outside the structural support exactly when both its row and its column are. For a reference, whose centres are Hermitian and whose radii are symmetric (Definition 5.3), row i of a block is structurally zero exactly when column i is.

Where D is an operator family read as a measurement against a sector map with blocks $s \in \mathsf{S}$, over an alphabet J, a block s of a snap $F = D'$ of D is *kept* when $F_j^{(s)} = D_j^{(s)}$ is decided exactly for every $j \in J$, and *moved* otherwise. Here D on block s is the data the snap is made from: the centres of an exact reference of the block where the snap runs on them, a producer's exact construction, or the data as they stand where the declarer states them with no model behind them. Where the declarer computes the data of the block from a model of it and the snap runs on that evaluation, the data the snap is made from are the model, of which the evaluation is no exact presentation, so no exact decision equates the snap with them and the block is moved, even where the snap leaves the evaluation unchanged. A snap with no moved block changes nothing.

The definition fixes what a snap produces and not how. The displacement is a record of the move, and it is the whole of what relates D' to D here.

Definition 5.7 (A reference handed beside a snap; a block evaluated off its snap)**.** Let F be a snap (Definition 5.5) of data D against a sector map with blocks $s \in \mathsf{S}$ (Definition 4.2), over an alphabet J, where D is an operator family read as a measurement, the data an instance states of it or a reference

of it, or a producer's exact construction of what it emits (Definition 4.1).

(i) Where D is an operator family read as a measurement and the declarer holds a reference G of it (Definition 5.3), the declarer *hands the reference beside the snap* on a block s when it passes to every consumer of F, together with F and altered by no step, the centre $C_j^{(s)}$, with entries in $\mathbb{Q}(i)$, and the radii $r_j^{(s)}$, rationals, of an entrywise enclosure of $G_j^{(s)}$ for every $j \in J$, with the identity of the reference and the claims that establish it. The reference so passed is the *handed reference*, and s is a *handed block*.

(ii) A block s of F is *evaluated off its snap* in a run when the record of the run shows that no number from which the run computes its key length reads $F_j^{(s)}$ for any $j \in J$, save in two places: the points at which those numbers are evaluated, which any computation may have chosen, a computation on F among them; and every value the run fixes before its first round other than the output length, the acceptance set and the transcript length λ_{EC} among them, whatever they were computed from.

6 Domination and its limits

Definition 6.1 (Domination of one operator family by another at block weights)**.** Let $\mathfrak{m}$ be a sector map with blocks $s \in \mathsf{S}$ of dimensions d_s (Definition 4.2), let J be a finite non-empty alphabet, and let $G = \{G_j\}_{j\in J}$ and $F = \{F_j\}_{j\in J}$ be families of block lists against $\mathfrak{m}$ whose blocks are Hermitian and positive semidefinite; their entries are arbitrary complex numbers. A family is *complete at* a real number c on block s when $\sum_{j\in J} G_j^{(s)} = c\,\mathbb{1}_{d_s}$. For weights $q = (q_s)_{s\in\mathsf{S}}$ with $q_s \in [0,1)$, the family G *dominates* F at q when

$$G_j^{(s)} - (1-q_s)\,F_j^{(s)} \succeq 0 \qquad \text{for every } j \in J \text{ and } s \in \mathsf{S}. \tag{6.1}$$

The weight q_s belongs to the block s and is the same for every outcome. On a block s, the *residual family* of the domination is $R_j^{(s)} := q_s^{-1}\big(G_j^{(s)} - (1-q_s)F_j^{(s)}\big)$ for $j \in J$ where $q_s > 0$, and $R_j^{(s)} := G_j^{(s)}$ where $q_s = 0$. For a Hermitian matrix X we write $\operatorname{supp} X$ for its range, the orthogonal complement of its kernel.

Lemma 6.2 (Elementary properties of domination)**.** Let $\mathfrak{m}$, J and the families $G = \{G_j\}_{j\in J}$, $F = \{F_j\}_{j\in J}$ and $E = \{E_j\}_{j\in J}$ of block lists against $\mathfrak{m}$ with Hermitian positive semidefinite blocks be as in Definition 6.1, and let $s \in \mathsf{S}$ be a block. Say that G dominates F at q_s on block s when (6.1) holds at that block.

(i) *(Upward closure.)* If G dominates F at q_s on block s and $q_s \le q'_s < 1$, then G dominates F at q'_s on block s.

(ii) *(Support.)* If G dominates F at some $q_s \in [0,1)$ on block s, then $\operatorname{supp} F_j^{(s)} \subseteq \operatorname{supp} G_j^{(s)}$ for every $j \in J$. Conversely, if $\operatorname{supp} F_j^{(s)} \subseteq \operatorname{supp} G_j^{(s)}$ for every $j \in J$, then G dominates F on block s at some weight in $[0,1)$.

(iii) *(Weight zero.)* If G and F are both complete at the same c on block s, then G dominates F at 0 on block s if and only if $F_j^{(s)} = G_j^{(s)}$ for every $j \in J$.

(iv) *(Composition.)* If G dominates F at q_s and F dominates E at q'_s on block s, then G dominates E on block s at $1-(1-q_s)(1-q'_s)$, which lies in $[0,1)$ and is at most $q_s + q'_s$.

(v) *(Classical post-processing.)* Let W be a finite non-empty alphabet and let $P_{w|j} \ge 0$ be real numbers for $w \in W$ and $j \in J$. Put $G'_w := \sum_{j\in J} P_{w|j} G_j$ and $F'_w := \sum_{j\in J} P_{w|j} F_j$, block by block. If G dominates F at q_s on block s, then G' dominates F' at the same q_s on block s. If moreover $\sum_{w\in W} P_{w|j} = 1$ for every j and G and F are complete at c on block s, then so are G' and F'. In particular, multiplying each G_j and F_j by the same number $p_j \ge 0$ preserves domination at the same weight.

(vi) *(Residual family.)* If G and F are both complete at c on block s and G dominates F at q_s there, then the residual family $R^{(s)}$ of Definition 6.1 has positive semidefinite elements, is complete at c, and satisfies

$$G_j^{(s)} = (1-q_s)\, F_j^{(s)} + q_s\, R_j^{(s)} \qquad \text{for every } j \in J. \tag{6.2}$$

Lemma 6.3 (Domination of a family that absorbs its residue in one element)**.** Let s be a block of dimension d_s, J a finite non-empty alphabet, $c > 0$, and $j^\star \in J$. Let $\{G_j\}_{j\in J}$ be Hermitian positive semidefinite $d_s \times d_s$ matrices with $\sum_{j\in J} G_j = c\,\mathbb{1}_{d_s}$, and let $\{F_j\}_{j\in J}$ be Hermitian matrices with

$$0 \preceq F_j \preceq G_j \quad (j \ne j^\star), \qquad F_{j^\star} = c\,\mathbb{1}_{d_s} - \sum_{j\ne j^\star} F_j. \tag{6.3}$$

Put $\Delta := \sum_{j\ne j^\star}(G_j - F_j)$, the *absorbed residue*.

(i) $\Delta \succeq 0$, $F_{j^\star} = G_{j^\star} + \Delta \succeq G_{j^\star}$, every F_j is positive semidefinite, and $\sum_j F_j = c\,\mathbb{1}_{d_s}$.

(ii) For $q \in [0,1)$, $\{G_j\}$ dominates $\{F_j\}$ at q on this block (Definition 6.1) if and only if

$$q\,G_{j^\star} - (1-q)\,\Delta \;\succeq\; 0. \tag{6.4}$$

For every $j \neq j^\star$ and every $q \in [0,1)$, $G_j - (1-q)F_j \succeq 0$ holds.

(iii) Some $q \in [0,1)$ satisfies (6.4) if and only if $\operatorname{supp}\Delta \subseteq \operatorname{supp} G_{j^\star}$; this holds in particular when $G_{j^\star}$ is positive definite.

(iv) Suppose $\operatorname{supp}\Delta \subseteq \operatorname{supp} G_{j^\star}$, let $\lambda > 0$ be the least non-zero eigenvalue of $G_{j^\star}$ (its least eigenvalue when it is positive definite) and let $\delta \geq \|\Delta\|$. If $G_{j^\star} \neq 0$, then (6.4) holds for every q with $\delta/(\lambda+\delta) \leq q < 1$. If $\Delta = 0$, it holds at $q = 0$.

Lemma 6.4 (Rationality obstruction to the domination of a lattice family)**.** Let $d \geq 1$, and read every vector of $\mathbb{C}^d$ and every $d \times d$ matrix in the standard basis. Call a subspace U of $\mathbb{C}^d$ $\mathbb{Q}(i)$-*rational* when it is spanned over $\mathbb{C}$ by vectors of $\mathbb{Q}(i)^d$ (the subspace $\{0\}$ is spanned by the empty set), and for a subspace V of $\mathbb{C}^d$ put

$$\mathcal{R}(V) \;\coloneqq\; \operatorname{span}_{\mathbb{C}}\big(V \cap \mathbb{Q}(i)^d\big), \tag{6.5}$$

the *largest* $\mathbb{Q}(i)$*-rational subspace of* V.

(i) $\mathcal{R}(V)$ is a $\mathbb{Q}(i)$-rational subspace contained in V, and it contains every $\mathbb{Q}(i)$-rational subspace contained in V.

(ii) For every $d \times d'$ matrix A with entries in $\mathbb{Q}(i)$, the range of A is $\mathbb{Q}(i)$-rational. In particular the support of a Hermitian $d \times d$ matrix with entries in $\mathbb{Q}(i)$ is.

(iii) Let $\{G_j\}_{j\in J}$ and $\{F_j\}_{j\in J}$ be Hermitian positive semidefinite $d \times d$ matrices over a finite non-empty alphabet J, every F_j with entries in $\mathbb{Q}(i)$, such that $G_j - (1-q)F_j \succeq 0$ for every j, for some $q \in [0,1)$. Then $\operatorname{supp} F_j \subseteq \mathcal{R}_j \coloneqq \mathcal{R}(\operatorname{supp} G_j)$ for every j.

(iv) If, in (iii), moreover $\sum_j G_j = \sum_j F_j = c\,\mathbb{1}_d$ with $c > 0$, then for every $j \in J$

$$q \;\geq\; \frac{\lambda_{\max}\big(P_j G_j P_j\big)}{c}, \tag{6.6}$$

where P_j is the orthogonal projector onto $\mathcal{R}_j^{\perp}$.

(v) In particular, if $\operatorname{supp} G_j$ contains no non-zero vector of $\mathbb{Q}(i)^d$, then $q \geq \lambda_{\max}(G_j)/c$, which equals $\operatorname{tr} G_j/c$ when G_j has rank one.

Lemma 6.5 (Positive semidefiniteness from an entrywise enclosure)**.** (i) Let X be a Hermitian complex $d\times d$ matrix, X_c a Hermitian $d\times d$ matrix with entries in $\mathbb{Q}(i)$, and $r_{il}\geq 0$ rationals with $|X_{il}-(X_c)_{il}|\leq r_{il}$ for all i,l. Let ρ be a rational with

$$\rho \;\geq\; \max_i \sum_l r_{il} \qquad \text{or} \qquad \rho\geq 0 \;\text{ and }\; \rho^2 \;\geq\; \sum_{i,l} r_{il}^2, \tag{6.7}$$

and $\mu\geq 0$ a rational. If $X_c-(\rho+\mu)\,\mathbb{1}_d\succeq 0$, then $X\succeq \mu\,\mathbb{1}_d$; in particular $X\succeq 0$, and X is positive definite when $\mu>0$. If $X_c-\rho\,\mathbb{1}_d$ is positive definite, then X is positive definite. Every hypothesis on X_c, r, ρ and μ is a property of finitely many rationals and Gaussian rationals.

(ii) Let $n_1,\dots,n_d$ be positive rationals and $D\coloneqq \operatorname{diag}(\sqrt{n_1},\dots,\sqrt{n_d})$, and let a_i,e_i be rationals with $0\leq e_i\leq a_i$ and

$$(a_i-e_i)^2\,n_i \;\leq\; 1 \;\leq\; (a_i+e_i)^2\,n_i. \tag{6.8}$$

Then $|a_i-n_i^{-1/2}|\leq e_i$. Let S and Y be Hermitian $d\times d$ matrices with entries in $\mathbb{Q}(i)$, put $X\coloneqq D^{-1}SD^{-1}-Y$, $A\coloneqq \operatorname{diag}(a_1,\dots,a_d)$, $X_c\coloneqq ASA-Y$ and

$$r_{il} \;\coloneqq\; \sigma_{il}\left(a_ie_l+e_ia_l+e_ie_l\right), \qquad \sigma_{il}\coloneqq |\operatorname{Re}S_{il}|+|\operatorname{Im}S_{il}|. \tag{6.9}$$

Then X, X_c and r satisfy the hypotheses of (i) on X, X_c and r, and the radii do not depend on Y.

(iii) Let X, X_c and r be as in (i), let F be a Hermitian positive semidefinite $d\times d$ matrix with entries in $\mathbb{Q}(i)$, and let $q\in[0,1]$ be rational. Then $X-(1-q)F$, $X_c-(1-q)F$ and r satisfy the hypotheses of (i) on X, X_c and r. If, for some ρ and μ as in (i), $X_c-(1-q)F-(\rho+\mu)\mathbb{1}_d\succeq 0$, then $X_c-(1-q')F-(\rho+\mu)\mathbb{1}_d\succeq 0$ for every rational $q'\in[q,1]$.

7 The exact island, enclosures and the tiers

Definition 7.1 (The exact island)**.** The *exact island* is the following set of computations of the engine, the declarations of its instances included.

(i) *The admission boundary*: every decision that data satisfy a condition their declaration states of them — among them the conditions of Definition 4.3, a trace, a column sum, an identity between declared operators and a lattice condition — taken by a consumer of declared data it reads, or by a declarer, an instance or a producer (Definition 4.1), of data it is about to declare; the decisions by which a snap establishes the conditions of its output (Definition 5.5) are among the

latter. So are, for a reference of an operator family that a declarer uses or hands beside its snap (Definition 5.3), the computation of the centres and radii of its entrywise enclosures, whose enclosure of the reference, (5.1), the claims of the module that constructs the reference establish, the decisions on those centres and radii and, for an exact reference, on its positivity and completeness; and, for any snap of an operator family read as a measurement, the decisions by which the snap keeps a block, and the decisions on which entries are structurally zero. It contains, further, a producer's construction, block by block, of the operator data it emits, and, for each scalar bound that a consumer replaces as in Definition 4.1 (iii), the decision that the replacement lies on the weakening side of the bound emitted.

(ii) *The acceptance side of the optimisation*, the honest model and the completeness budget being declared as exact rational data: the construction of an acceptance set from them; the exact establishment that the optimiser's acceptance-side iterate lies in the acceptance set it reads, for its initial point and for each update as a convex combination, with exact rational weights, of points so established; and the exact certification of the minimum of a linear functional over that set, or of a lower bound on that minimum. The acceptance set the optimiser reads is the set so declared: the constructed set as its producer emits it, with each scalar bound that a consumer has replaced as in Definition 4.1 (iii) at its replaced value.

(iii) *The feasible side of the optimisation*: the certification of a lower bound on the minimum of a linear functional over the set the optimiser's linear oracle is asked to minimise over, formed from the functional, from the data of that set and from multipliers, each given to it as values, together with the bounds on least eigenvalues that the certification computes on the way.

The decision whether data an instance is about to declare lie within the declared tolerance (Definition 5.1) is not on the exact island. The optimiser's iterate apart from its acceptance side, the optimiser's oracle, the objective and its gradient lie outside the exact island, and so does a binary64 computation that only proposes a candidate, such as a point or a multiplier, for a decision or a certification of the island. A certification of (iii) holds of the functional and the data as they are given to it, and not of the quantities that a binary64 computation of them approximates.

Lemma 7.2 (Least-eigenvalue lower bound from an approximate eigenbasis)**.** Let $n \geq 1$, let R be a Hermitian complex $n \times n$ matrix, and let

$$\widetilde{R} \coloneqq \begin{pmatrix} \operatorname{Re} R & -\operatorname{Im} R \\ \operatorname{Im} R & \operatorname{Re} R \end{pmatrix} \in \mathbb{R}^{2n \times 2n}, \tag{7.1}$$

the real and imaginary parts taken entrywise. Let V be any real $2n \times 2n$ matrix, put

$$B \coloneqq V^{\mathsf{T}} \widetilde{R} V, \qquad E \coloneqq V^{\mathsf{T}} V - \mathbb{1}_{2n}, \qquad \gamma \coloneqq \min_{1 \le i \le 2n} \Big(B_{ii} - \sum_{l \neq i} |B_{il}| \Big),$$

and let θ be a real number with $0 \le \theta < 1$ and $\sum_{i,l} E_{il}^2 \le \theta^2$.

(i) $\widetilde{R}$ is real symmetric, and $\lambda_{\min}(\widetilde{R}) = \lambda_{\min}(R)$.

(ii) V is invertible, and

$$\lambda_{\min}(R) \;\ge\; \begin{cases} \gamma/(1+\theta), & \gamma \ge 0, \\ \gamma/(1-\theta), & \gamma < 0. \end{cases} \tag{7.2}$$

(iii) If the entries of R lie in $\mathbb{Q}(i)$, the entries of V are rational and θ is rational, then $\widetilde{R}$, B and E have rational entries, γ is rational, the hypothesis on θ is a comparison of rationals, and the right-hand side of (7.2) is a rational number.

(iv) Applied to $-R$, (ii) bounds the largest eigenvalue of R, which is $-\lambda_{\min}(-R)$, from above.

Definition 7.6 (The tiers of a reported key rate)**.** A *tier* of a reported key-rate number is one of the following three labels.

(i) *estimate*: the number is the value of the objective at the point an optimiser returned. It is reported as an estimate of the minimum of the objective and is claimed as no bound.

(ii) *first-order*: the number is a lower bound on the minimum of the objective, established by a claim under hypotheses that claim names.

(iii) *certified*: the number is a lower bound on the minimum of the objective of the declaration it is formed on, established by a claim under hypotheses that claim names, and every arithmetic step between that declaration's data and the number is exact or *rigorously enclosed*: a computation is rigorously enclosed when each of its operations returns a set of numbers, such as an interval or a ball, that contains the exact result of the operation at every value the sets of its inputs contain, so that the set it ends with contains the exact value, at the exact data, of the quantity it computes. None of those hypotheses is an assumption about how a number was computed, that is, none is an assumption of the category numerical-trust.

The tiers are ordered, estimate below first-order below certified, and a tier is *stronger* than the tiers below it. A certified number is a first-order number as well. There is no other tier.

8 The snapped-measurement assumption

Assumption 8.1 [**A-SNAP**] (The device measures the snapped family where the run reads the snap)**.** Let F be a snap (Definition 5.5) of an operator family D read as a measurement, against a sector map with blocks $s \in \mathsf{S}$ (Definition 4.2), over an alphabet J, with D the data the snap is made from: the data as an instance states them, or, where the instance computes those data from a model of the family, that model, a reference of the family (Definition 5.3), or a producer's exact construction of what it emits (Definition 4.1). Let $B \subseteq \mathsf{S}$ be the set of blocks that the record of the run names as resting on this assumption: the moved blocks of F (Definition 5.5), those on which the record does not show, by an exact decision, that $F_j^{(s)} = D_j^{(s)}$ for every $j \in J$, that are not evaluated off its snap (Definition 5.7). A kept block and a block evaluated off its snap do not belong to B. Every other moved block belongs to B, whatever its cause; among them are the blocks on which the snap gives an element support outside the structural support of $D_j^{(s)}$ (Definition 5.5).

(i) Where D is the data an instance states as the measurement of a device, the model it computes them from, or a reference of it, then on every block $s \in B$ the block on s of the measurement the device performs, read against the sector map, is the family $\{F_j^{(s)}\}_{j \in J}$ with the entries recorded for the snap. On such a block, any statement of another assumption of the run that a model describes that measurement is not asserted, and this clause holds in its place.

(ii) Where D is a producer's exact construction of what it emits, every conclusion about the device that the claims of the run draw from the emitted family, under their hypothesis that it is the producer's exact construction, holds when the emitted family is $F^{(s)}$ on every block $s \in B$.

Engine requirements

9 Admission

Requirement 9.1 (Exact admissibility at every module boundary). Declared operator data are exactly admissible at every module boundary (Definition 4.1). Precisely:

(i) every declared operator family that a consumer reads as a measurement, that is, as a positive operator-valued measure or as a family with a declared completeness constant, is exactly admissible with that constant for the sector map the consumer holds (Definition 4.3);

(ii) every other declared operator, and every declared matrix or vector whose entries are declared values, satisfies exactly, on those values, each condition its declaration states of it, such as Hermiticity, positivity, a trace, a column sum or an identity with another declared operator, and every sum such a condition reads rounds nowhere in its declared order (Definition 3.3).

This is established before the data cross the boundary. An instance establishes it by constructing or snapping what it declares (Requirement 10.4), or by passing on unchanged, from a supplier's output (Definition 4.1 (ii)), data whose conditions the consumer decides exactly on entry, that decision then establishing it. A producer establishes it by constructing exactly what it emits and by deciding exactly, before emitting, each condition that its construction does not already establish, a sum that rounds nowhere among them; for a measurement family that decision is the one of Requirement 11.3. Data that fail it do not cross, because a consumer that finds one of its admission predicates violated refuses them (Requirement 9.2).

Requirement 9.2 (Exact decision by every consumer). Every consumer (Definition 4.1) decides every admission predicate it states of declared data exactly, on the declared values, and none more loosely than exactly: no consumer has an input through which a predicate on declared data would be decided at a nonzero defect, and none admits data on the ground that they lie within a tolerance of data it would admit. A consumer that finds such a predicate violated refuses, with a machine-readable reason naming the predicate that failed. It repairs nothing it was handed, and it neither snaps nor otherwise alters declared data of another declarer, with one exception: it may replace a scalar bound that a producer emits by that bound rounded to a binary64 number in a direction that the producer's own claims license as weakening it. The direction in which each bound so replaced was rounded is recorded where the consumer's reading of the emission is specified, nothing else of the emission is altered, and every predicate on such a bound is decided exactly on its replaced value. The declared tolerance therefore widens nothing that any consumer admits; it bounds only how far an instance may move its own declaration (Requirement 10.1).

Requirement 9.3 (Summation order declared with the data). Where a predicate on declared data is a sum, the order in which the sum is accumulated is part of the declared data: it is declared with the alphabet that indexes the sum, as that alphabet's declared order (Definition 3.3), or with an alphabet containing it, whose declared order it then carries by restriction; and it travels with the artefact that carries the data. Where a declared order arrives both with the alphabet and with an alphabet containing it, the two are one order: the first is the restriction of the second. A consumer that receives two such orders that differ takes the order of the containing alphabet,

decides the predicate in its restriction, and refuses nothing for the difference. Every consumer that decides the predicate, and every declarer that decides it of data it is about to declare (Definition 4.1), decides it in that order, and no consumer substitutes an order of its own; a supplier deciding its own conditions at slacks declared to it is not bound by this. Only where no declared order arrives with the alphabet or with an alphabet containing it is the declaration's own listing of the alphabet taken as its declared order. Where such a predicate reads the trace of a declared matrix, the trace is accumulated in increasing row index.

10 The tolerance, norms and snapping

Requirement 10.1 (One declared tolerance per instance). Each instance declares exactly one declared tolerance τ_{num} (Definition 5.1) in its run configuration, and τ_{num} has no default: a run whose configuration does not declare it, or declares a value that is not a binary64 number in $[0, 1)$, is refused before anything is constructed. τ_{num} is the ceiling on the defect an instance may repair in what it declares. It binds the data an instance is about to declare other than numbers of a supplier's output that it passes on unchanged (Definition 4.1 (ii)); those it neither measures nor repairs, and their consumer decides their conditions exactly on entry (Requirement 9.1). Of the data it binds that have no exact reference on whose centres the snap runs (for those, see the next paragraph), the instance measures in binary64 the defects (Definition 5.1) and, for each block read as positive semidefinite, the least eigenvalue of its Hermitian part, each on the data as they stand before any step of the snap, and records those measured numbers. Data with a measured defect above τ_{num} or a measured least eigenvalue below $-\tau_{\mathrm{num}}$ are refused as a modelling error; the others may be snapped (Definition 5.5), the snap being recorded with the measured defects it corrected (Requirement 10.4). Near τ_{num} this decision on measured numbers may differ from the predicate *within the declared tolerance* of Definition 5.1, whose sums are exact, and no bound on that difference is kept. This decision is not on the exact island; the exact decisions are those by which the snap establishes the conditions of its output (Definition 7.1 (i)).

An operator family that an instance snaps on the centres of an exact reference (Definition 5.3), as Requirement 10.5 (ii) states, stands before the snap as that reference, which is Hermitian, positive semidefinite and complete by the exact decision on its entries, and never by a binary64 evaluation. Its defects are therefore zero and no block of it falls below the positivity floor: τ_{num} decides nothing for it, and the instance records the defects as zero on that ground rather than on a binary64 measurement. τ_{num} keeps its role for the data it binds on which the snap runs as they stand, an evaluation of a model whose reference has a radius that is not zero included, since the snap

does not run on such a reference.

A probability vector that an instance prepares from ordinary numerical inputs before declaring it is decided instead by the rule of the input preparation. It is refused unless its normalisation defect, computed exactly on the inputs as given, is at most τ_{num}, or the sum of the rounding cells of its inputs contains 1: a binary64 input contributes the rounding cell it denotes under the preparation's documented representation policy, and an input given exactly, as a decimal or a fraction, contributes the one point that is its value. A value below 0 or above 1 is refused. The preparation records the defect, the cells and which of the two grounds admitted the vector. Its snap then runs at a *guard*, the larger of τ_{num} and the completeness residue the snap measures on the binary64 conversion of the inputs, which may exceed τ_{num}; the guard is recorded, and it admits no vector the rule has refused.

Requirement 10.2 (Scope of the declared tolerance). The declared tolerance governs the repair of an instance's own declared data (Requirement 10.1) and nothing else; in particular no consumer reads it (Requirement 9.2).

(i) A set defined by declared data is the exact set those data define, and the declared tolerance relaxes none of its defining conditions. A consumer that certifies a supplied point as a member of such a set certifies membership in the exact set, deciding it exactly on the declared values of the point. A check of a supplied point at a window of the module's own certifies no membership in such a set, and its window is a number of that module, as in (ii).

(ii) The declared tolerance does not govern a module's arithmetic on computed objects (Definition 4.1): a window at which a module checks a computed object, or within which it repairs one, is a number that module declares and records in its own right, and it is never the declared tolerance.

(iii) The declared tolerance is a numerical tolerance and not a device-uncertainty budget. A declared number that bounds a physical quantity, such as a tail budget, is declared separately and is never derived from the declared tolerance. A slack that an instance declares to a supplier (Definition 4.1 (ii)) for the supplier's own conditions is a declared number of its own. One that bounds the error of the supplier's floating-point evaluation of a quantity, such as a slack on a computed eigenvalue (Requirement 12.1), is never derived from the declared tolerance; any other is derived from it, if at all, only as Requirement 10.3 says.

Requirement 10.3 (Explicit conversion between norms). No bound stated in one norm is read as a bound in another. The defects of Definition 5.1 are entrywise maximum moduli and the positivity floor is a bound on a least eigenvalue, and neither is converted, implicitly, into the other or into a bound in any other norm. A declared number that is derived from the declared tolerance, or from any bound stated in one norm, for use as a bound in another norm is derived explicitly by the party that declares it, with the conversion factor between the two norms concerned written down (for the operator norm and the entrywise maximum modulus, that of Lemma 5.2 (i)), and it is recorded together with that factor.

Requirement 10.4 (Snapping by the declarer only). Declared data are snapped (Definition 5.5) only by their declarer, at most once, and before any consumer reads them: an instance snaps what it declares about its device and its protocol, on the admission that Requirement 10.1 states (for an operator family snapped on the centres of an exact reference, the decisions on the reference that Requirement 10.5 (i) lists; for a probability vector prepared from numerical inputs, the rule of the input preparation; otherwise within the declared tolerance), and a producer snaps only its own emission. No party snaps data it was handed, and no party otherwise alters them, with the one exception that Requirement 9.2 states and limits: a consumer may replace a scalar bound that a producer emits by that bound rounded in a direction that the producer's own claims license as weakening it, recording the direction. Every snap is recorded: an instance's in the record of the run, with the defects it measured, or decided to be zero, before the snap (Definition 5.1), the displacement and the rule it applied, and a producer's in its emission, with the disclosures of Requirement 11.3. The record of a snap of an operator family read as a measurement carries, besides, the identity of every reference the declarer uses or hands beside the snap (the model, the exact parameters from which it is formed, and its entrywise enclosures where any radius is not zero, with the claims that establish them; Definition 5.3); the *reference kind* of the snap per block (*exact*, where the snap runs on the centres of an exact reference of the block, or *none*, where it runs on the data as they stand); whether the run declares that enclosed references are not used; the status of every block, kept or moved (Definition 5.5); the steps of the procedure that made each moved block (Requirement 10.6); and the flag `support_not_preserved` of every block snapped by the last resort (Requirement 10.6 (iv)). Where the declarer hands the reference beside the snap (Definition 5.7 (i)), the record names the handed blocks, with the kind of the reference handed on each (*exact*, every radius zero, or *enclosed*, with the largest radius of the block and the largest radius of the enclosures of irrational inputs (square roots, trigonometric values) from which the constructing module computed the centres exactly, as its claims state it), and, for every block on which it hands none, the cause of Requirement 10.5 (iii)(a).

The record of the run names every moved block of the snap that is evaluated off its snap (Definition 5.7 (ii)), and every moved block that rests on Assumption 8.1, with its cause, group and fix (Requirement 10.5 (iii)); a kept block is named as kept and in neither list. Handing the reference alters neither the snap nor the reference. A producer that declares a lattice exponent re-establishes its lattice at each emission whose exact construction lies off it, by a snap of that emission or by refusing it (Requirements 11.1 and 11.3); a producer that declares no lattice exponent snaps nothing.

Requirement 10.5 (Snaps of a measurement family: kept, or moved and disclosed). (i) *The reference.* Before it snaps an operator family read as a measurement, an instance of what it declares or a producer of its own emission (Definition 4.1), the declarer decides exactly, of every reference of the family it uses or hands beside the snap (Definition 5.3), that every centre is Hermitian with entries in $\mathbb{Q}(i)$ and that the radii are non-negative rationals with $r_{il} = r_{li}$, of every factor enclosure it carries (Definition 5.4) that the centre of its matrix box has entries in $\mathbb{Q}(i)$, that its radii are non-negative rationals and that its core is diagonal with positive rational entries, and, for an exact reference, that it is positive semidefinite and complete at its constant. For a reference with a radius that is not zero, three properties are not decided by the declarer: that its enclosures enclose it, (5.1), and that it is positive semidefinite and complete at its constant. They are established by the claims of the module that constructs the reference, which the record names, and its centres and radii are computed in exact rational arithmetic on the exact island (Definition 7.1 (i)); a binary64 evaluation establishes none of them. A reference of which any of them is not so established is neither used nor handed. Nor is it decided by the declarer, of any reference, that a block on which it carries a factor enclosure is a member of it, (5.3): that too is established by the claims of the module that constructs the reference, which the record names, and the centres and radii of the factor enclosure are computed in exact rational arithmetic or by a rigorously enclosed computation (Definition 7.6). A factor enclosure of which either is not so established is not carried, and the reference is read without it. A producer's exact construction, where it is exactly Hermitian, positive semidefinite and complete, is an exact reference of its emission. An instance that holds a reference so established hands it beside its snap (Definition 5.7 (i)) on every block, save that, in a run whose configuration declares that enclosed references are not used, by the option `reference_enclosure`, it hands only the blocks every radius of which is zero.

(ii) *The snap.* Every such snap is made block by block. A block is kept, $F^{(s)} = G^{(s)}$, where the declarer establishes an exact reference G of

that block, against the sector map of the block alone, decided as (i) states, every element of which is a lattice value at the lattice exponent k (Definition 3.1 (iv)), and where exact admissibility at the constant c (Definition 4.3) and the lattice condition at k, decided last on the block, hold. For a producer the block kept is the one its own claims keep. Every other block is made by a procedure that meets Requirement 10.6 and aims at fidelity to the data it starts from rather than at any other property: an instance's is one procedure for every such block, the support-safe steps specified with the shared numerical code, run on the centres of an exact reference of the block where the declarer establishes one, and otherwise on the data as they stand, admitted within the declared tolerance; a producer's is the procedure its own claims state. Where that procedure yields on a block no data that satisfy the conditions of Definition 5.5, the block is snapped by the last resort of Requirement 10.6 (iv), which the record flags; where the last resort too yields no such data, the declarer refuses the data, naming the block and the condition that failed. Every block made by this procedure or the last resort is moved (Definition 5.5) unless the snap is decided exactly to equal on it the data it is made from.

(iii) *Resting.* A moved block *rests on Assumption 8.1* when it is not evaluated off its snap (Definition 5.7); a kept block and a block evaluated off its snap do not. The record of the run names every block that rests on the assumption with its cause, its group and a fix. The group is *last resort* for a block snapped by the last resort, whose fix is a model that includes the noise the device has, so that every element of the block is positive definite, together with the fix of its cause; otherwise it is the group of its cause:

(a) *no usable reference*: no reference is handed on the block, because the declarer holds no reference of the family, because a reference fails the decisions of (i) or a property that (i) leaves to the constructing module is not established, or because the run declares that enclosed references are not used and a radius of the block is not zero; for a class of a producer's snap of its own emission, because the producer forms no companion that carries its exact construction past the snap, a *bypassing companion*. The fix is a model of the family with a reference that the run uses, and for the last of the instance's causes an exact reference, or a run that uses enclosed references; for a producer's class, declared numbers on which its exact construction stays on the lattice, so that the class is kept;

(b) *reference bracket withdrawn*: a reference is handed on the block, or, for a producer's class, a bypassing companion is formed, and a

companion carrying the reference on the block, or bypassing the class, is evaluated, but the key length the run reports is formed from a number on the snapped declaration, for one of three causes. Either no evaluated point contributes a bound on the companion, so the run's first-order number is formed on the snapped declaration (Requirement "The first-order bound evaluated on the reference companion" (meat-core/frank-wolfe/req:bracket-on-reference)); or the certified evaluation finds the two enclosures of a block the companion carries inconsistent, the bracket on the companion being withdrawn and the run's first-order number formed on the snapped declaration, the cause `factor_enclosure_inconsistent` (Requirement "The certified evaluation of the first-order bound" (meat-core/frank-wolfe/req:certified-evaluation) (iv)); or a point contributes, but the certified evaluation on the companion certifies no point and the key length is formed from a number certified on the snapped declaration in its place, the cause `companion_not_certified` (Requirement "The certified evaluation of the first-order bound" (meat-core/frank-wolfe/req:certified-evaluation) (i)). The fix is the one the reasons recorded at the evaluated points of the companion name: the withdrawal reasons for the first cause and the reasons of the certified evaluation for the third; for the second, no change of the declaration, an inconsistency being a defect of the engine in the modules that carry the enclosures, so the fix says that and that the defect be reported;

(c) *reference bracket not evaluated*: a reference is handed on the block, or, for a producer's class, a bypassing companion is formed, but no companion carrying the reference on the block, or bypassing the class, is evaluated, because none is formed, compiled or admitted. The fix is the one the reason recorded for the companion names.

(iv) *The strict option.* A run whose configuration declares the *strict option* `strict_snaps` refuses wherever a moved block would rest on Assumption 8.1, naming the block, its cause, its group and the fix, and reports no key length that rests on the assumption. It refuses as soon as the resting is decided: a declarer when it snaps a moved block on which it hands no reference, a producer when it snaps a moved class that no bypassing companion it forms carries past the snap, and the run, after its search and before any key length is reported, where a moved block or class is not evaluated off its snap. A strict run forms no certified number on the snapped declaration in place of a companion's (Requirement "The certified evaluation of the first-order bound" (meat-core/frank-wolfe/req:certified-evaluation) (i)), so the cause `companion_not_certified` of (iii)(b) does not arise in it, and it refuses on the other two. A strict run none of whose moved blocks rests

on the assumption proceeds as one that does not declare the option. In a run that does not declare it, the run proceeds; its record carries, for every snap, the status of each block, kept or moved, the flag of every block snapped by the last resort, the blocks that rest on the assumption with their causes, groups and fixes, and Assumption 8.1 among the assumptions the run rests on, naming those blocks, so that a report on the key rate can flag that the rate rests on a snap. A run none of whose blocks rests on the assumption does not rest on it.

Requirement 10.6 (Support-safe snaps of a measurement family). Every snap F at a lattice exponent k of an operator family D read as a measurement (Definition 5.5), against a sector map with blocks $s \in \mathsf{S}$ (Definition 4.2) and with constant c, by an instance of what it declares or by a producer of its own emission, with D the data as they stand or a reference of the family (Definition 5.3), satisfies (i) to (iii) on every block s and for every outcome j, save on a block snapped by the last resort of (iv).

(i) *No revival.* The structural support of $F_j^{(s)}$ is contained in that of $D_j^{(s)}$. In particular, if $D_j^{(s)}$ is structurally zero, then $F_j^{(s)} = 0$; and for every index i at which both row i and column i of $D_j^{(s)}$ are structurally zero, row i and column i of $F_j^{(s)}$ are zero, which for a reference is every index whose row is structurally zero.

(ii) *Lift per coordinate.* A step of the snap that changes the elements in order to make them positive semidefinite does one of two things. Either it adds to $F_j^{(s)}$ only a matrix $\sum_i t_i e_i e_i^*$ with $t_i \geq 0$, e_i the i-th coordinate vector of the block, and $t_i > 0$ only for indices i in the structural support of $D_j^{(s)}$; or it replaces the family on the block by $(1-w)E + wT$. Here $w \in (0,1)$ is rational; E is the family on the block as the preceding steps of the snap left it or, where every entry of $D^{(s)}$ is a Gaussian rational, $D^{(s)}$ itself; and T is a family complete at c on the block whose element $T_j^{(s)}$ is, for every j, a diagonal matrix with non-negative entries that are non-zero only at indices in the structural support of $D_j^{(s)}$.

(iii) *Residue.* A step of the snap that adds to an element what the sum of the elements lacks of $c\,\mathbb{1}_{d_s}$, or a part of it, adds it to $F_j^{(s)}$ only where the structural support of $D_j^{(s)}$ contains every index i such that row i or column i of what it adds has a non-zero entry.

(iv) *Last resort.* Where a procedure meeting (i) to (iii) yields on a block s no family that satisfies the conditions of Definition 5.5, the snap may, on that block alone, replace the family by $(1-w)E + wU$ and

then restore the lattice condition and completeness; (i) to (iii) do not bind on that block. Here $w \in (0, 1)$ is rational; E is the family that procedure produced on the block or, where every entry of $D^{(s)}$ is a Gaussian rational, $D^{(s)}$ itself; and U is a family complete at c on the block whose element $U_j^{(s)}$ is, for every j, a positive rational multiple of $\mathbb{1}_{d_s}$, or zero where $D_j^{(s)}$ is structurally zero. The record of the snap flags the block `support_not_preserved`, since an element may then have support where $D_j^{(s)}$ is structurally zero.

11 Emission

Requirement 11.1 (The declared lattice condition). (D-LAT) Let k be the one lattice exponent (Definition 3.1 (iv)) at which a declarer declares operator data. Every entry of every operator of those data is a lattice value at k: the real and the imaginary part of the entry are integer multiples of 2^{-k} and have modulus at most 1. Each entry is then exactly a binary64 number, by Lemma 3.2 (i).

Operator data are declared at a lattice exponent k, and (D-LAT) binds them, in these cases and no other: the data an instance snaps at k (Definition 5.5); the data an instance declares to a consumer that admits declared data only at a declared lattice exponent k; and every operator that a producer which declares a lattice exponent k emits (Definition 4.1). Where data of the second case are numbers of a supplier's output that the instance passes on unchanged (Definition 4.1 (ii)), the instance does not establish (D-LAT) of them: the consumer's exact decision on entry establishes it, and the consumer refuses data that fail it (Requirement 9.1). A producer that declares no lattice exponent has no obligation under (D-LAT).

Requirement 11.2 (Rounding direction of emitted bounds). (D-DIR) Every emitted scalar bound is rounded onto the lattice in the direction that weakens the claim it carries, never to nearest: a lower bound *down*, an upper bound *up*, a constraint constant in the direction that enlarges the feasible set for its declared relation. The direction is recorded per bound, and where a family's completeness depends on a rounded weight the complementary quantity is *recomputed* from the rounded weights, never rounded independently.

It binds every producer that declares a lattice exponent k (Definition 4.1), for each scalar bound it emits, the lattice being that of the lattice values at k (Definition 3.1 (iv)); the claim that states the emission names the direction of each such bound. A producer that declares no lattice exponent, and that constructs an emitted scalar bound exactly and then rounds it to a binary64 number, rounds it in the same direction, never to nearest.

Requirement 11.3 (Exact verification before emission). (D-EMIT) Every producer (Definition 4.1) emits an operator family as a measurement only after establishing, of the numbers it will emit and by exact arithmetic on them: (i) the sector map is the consumer's, decided and never measured; (ii) exact Hermiticity; (iii) exact positive semidefiniteness, by a rational LDL^* with the zero-pivot rule or an equivalent exact decision, never by a floating-point eigenvalue; (iv) exact completeness per sector block, as an identity of the emitted numbers; (v) the completeness sum in the declared order rounds nowhere in binary64, and, where it emits Alice's measurement $\{M_x\}$ and her marginal τ_A of a protocol declaration, so do the completeness sum of the one and the trace of the other; (vi) where it declares a lattice exponent k, Requirement 11.1 of every entry at k. Conditions (i)–(v) of a family are its exact admissibility with constant 1 for the sector map the consumer holds (Definition 4.3), and the clause of (v) on $\{M_x\}$ and τ_A binds only a producer that emits them.

A producer that cannot establish the conditions that bind it *refuses, naming the predicate*, save in the one case below; it repairs nothing it was handed. A producer that declares a lattice exponent k may, where the family it constructed fails any of (ii)–(iv) or (vi), instead *declare* a nearby family established under all six: a snap at k of its own emission (Definition 5.5), made as Requirement 10.5 states. Its moved blocks are related to its exact construction by no claim; a run that consumes the emission evaluates the numbers its key length rests on with the exact construction in their place where it can, and otherwise rests, on the blocks that Requirement 10.5 (iii) names, on Assumption 8.1 (ii): what it concludes from the emitted family holds of the snapped family. A run that declares the strict option of Requirement 10.5 (iv) refuses there instead. The producer must disclose that the emitted numbers are a declaration and not the constructed ones, the measured per-block displacement, the rounding direction of every weight that it rounds in constructing the emission, and, separately, the status of every block, kept or moved, and the flag of every block snapped by the last resort of Requirement 10.6 (iv). A producer that declares no lattice exponent declares no nearby family.

12 Arithmetic and tiers

Requirement 12.1 (Exact arithmetic on the island, unbudgeted rounding elsewhere). Every decision, certification and construction of the exact island (Definition 7.1) is computed in exact rational arithmetic on the values it is given, so that what it decides, certifies or constructs holds of those values exactly. A candidate proposed by binary64 arithmetic, such as a point or a multiplier, enters the exact island only as a value given to such a decision or certification, and nothing is established of it by the arithmetic that proposed

it. Outside the exact island the engine may compute in binary64, and it neither bounds nor budgets the rounding of those computations: no module other than a supplier tracks a floating-point error budget, and no result of the engine rests on one except as follows. A supplier (Definition 4.1 (ii)) may allow for the error of one of its own floating-point evaluations by a slack that an instance declares to it, under an assumption of its own claims, which it does not verify, that the slack bounds that error on the side that would make its output unsound. A result that rests on such an evaluation rests on that assumption, and the supplier carries the assumption, with its slack, in its output. A computation outside the exact island that is rigorously enclosed (Definition 7.6) is neither: each of its operations bounds its own rounding, so what it establishes holds of the exact quantity and no allowance is assumed to bound its error. The engine forms every number it reports at the certified tier from such computations and from the exact island alone.

Requirement 12.2 (Reporting of the tiers). Every run that reports a key rate reports it at every tier of Definition 7.6 that it reaches: the estimate always, the first-order number wherever the hypotheses of the claim that establishes it held, and the certified number wherever the requirement of the solver that forms it reports it at that tier, which may withhold the tier from a number that meets Definition 7.6 (iii) but rests on an upstream premise; together with the field `reported_tier`, which names the strongest tier reached, the tier that carries the run's claim. Where the first-order number is reported, the number at that strongest tier is the headline key rate, and the key length the run reports is formed from it; a number at a weaker tier is recorded beside it. No module reports a number under a tier stronger than the checks that number has passed support, and a request naming a tier that Definition 7.6 does not list is refused as an unknown tier.

References

[1] Marco Tomamichel. Quantum Information Processing with Finite Resources — Mathematical Foundations. *SpringerBriefs in Mathematical Physics, Vol. 5, Springer (2016)*, 2016.